

Project ASTRA — Personal AI Desktop Agent

Goal: Build a personalized AI desktop assistant capable of voice control, browser automation, PC task execution, file management, and autonomous workflows.

Phase 0 — Foundation Setup

Objectives

- Master JavaScript/TypeScript fundamentals
- Learn async programming and APIs
- Prepare development environment

Tools

- Node.js
- VS Code
- Git and GitHub
- Postman

Learn

- Promises and async/await
 - REST APIs
 - Filesystem operations
 - Environment variables
-

Phase 1 — Local AI Integration

Objectives

- Run local AI models
- Connect Node.js with AI model APIs

Tools

- Ollama
- Llama 3
- Mistral

Learn

- Prompt engineering
 - Model APIs
 - System prompts
-

Phase 2 — Tool Calling

Objectives

- Allow AI to execute actions

Tools

- `child_process`
- Custom tool registry

Learn

- Command routing
 - Safe execution
 - Tool schemas
-

Phase 3 — Browser Automation

Objectives

- Automate browser tasks

Tools

- Playwright
- Puppeteer

Learn

- DOM interaction
 - Selectors
 - Automation timing
-

Phase 4 — Voice Assistant

Objectives

- Enable voice commands

Tools

- Whisper
- Web Speech API

Learn

- Speech-to-text
 - Text-to-speech
 - Wake word systems
-

Phase 5 — Memory and RAG

Objectives

- Add memory and document recall

Tools

- SQLite
- MongoDB
- ChromaDB

Learn

- Embeddings
 - Vector databases
 - RAG
-

Phase 6 — Autonomous Workflows

Objectives

- Create proactive automated routines

Tools

- Node Cron
- Workflow engine

Learn

- Task scheduling
 - Retry systems
 - Workflow orchestration
-

Recommended Tech Stack

Category	Technology
Frontend	React / Next.js
Backend	Node.js + TypeScript
Local AI	Ollama
Models	Llama 3 / Mistral
Automation	Playwright
Voice	Whisper
Memory	SQLite / MongoDB
Vector DB	ChromaDB
Framework	LangChain JS

Important Advice: Start small and build incrementally. Focus first on reliable automation, then add AI reasoning, memory, and autonomy gradually.